

PROJECT AND TEAM INFORMATION

Project Title

Satellite Health Monitoring and Collision Avoidance

Student / Team Information

<i>Team Name:</i> <i>Team Id:</i>	<i>Sentra</i> <i>DSCP-III-2026-T004</i>
Team member 1 (Team Lead)	<i>Chhavit – 2510011425</i> <i>2510011425@geu.ac.in</i> 
Team member 2	<i>Jain, Harsh –</i> <i>2510370400</i> <i>2510370400@geu.ac.in</i> 
Team member 3	<i>Kumar, Satyam –</i> <i>2510380092</i> <i>2510380092@geu.ac.in</i> 
Team member 4	<i>Ojha, Sudhanshu –</i> <i>2510011648</i> <i>2510011648@geu.ac.in</i> 

PROPOSAL DESCRIPTION

Motivation

Space is becoming increasingly crowded with satellites and space debris, which makes both collision avoidance and satellite health monitoring important. A satellite can face an external threat such as approaching debris while also having internal problems such as low battery, low fuel, software errors, or a damaged thruster. If every problem has to be detected and handled manually from Earth, the response may take time. Our project aims to simulate a satellite that can monitor its own health, detect nearby collision threats, and make an appropriate response based on both the threat and its own condition.

State of the Art / Current solution

Today, space agencies use ground-based systems to track satellites and space debris and identify possible close approaches. When a collision risk is detected, operators analyze the situation and may send commands to the satellite for an avoidance maneuver. At the same time, satellite health is monitored using information such as battery level, temperature, fuel, and subsystem status. As the number of satellites increases, continuously monitoring both collision threats and satellite health becomes more difficult. Our project explores a simplified autonomous approach where the satellite can evaluate its surroundings and its own health before deciding how to respond.

Project Goals and Milestones

Our main goal is to build a C++ simulation of autonomous satellites that monitor their own health, detect nearby collision threats, prioritize those threats using data structures, and choose a suitable simulated response based on both collision risk and satellite health.

Milestones:

- 1. Core Foundation:** Create the basic Object-Oriented setup for satellites tracking position, fuel, and battery.
- 2. Specialized Types:** Add different types of satellites using Inheritance.
- 3. Space Grid (DSA):** Build the 3D grid system using an AVL Tree so satellites only look for debris in their current area.
- 4. Threat Priority (DSA):** Use a Min-Heap to prioritize nearby collision threats so that the most urgent threat can be considered first by the satellite's decision system.
- 5. Chaos Engine:** Generate random events such as battery drain, software faults, sensor problems, thruster failures, and new debris to test the satellite's health monitoring, recovery, and collision avoidance.
- 6. Logging:** Save every action to a blackbox text file.

Project Approach

Because this is a combined Project-Based Learning (PBL) assignment, we will build this project in C++ by combining Object-Oriented Programming (OOP) and Data Structures and Algorithms (DSA).

OOP Topics Used:

** **Classes and Objects:** Every satellite and piece of debris is built as a self-contained object.*

** **Inheritance:** We use a main base class called Satellite that handles basic flying and fuel. From there, we create specific types like CommunicationSatellite and ImagingSatellite.*

** **Polymorphism:** Each specific satellite has its own unique way to run health checks (using virtual functions), allowing them to self-heal differently.*

** **Encapsulation:** Important data like fuel levels and software health are locked inside the classes so outside code cannot accidentally change them.*

DSA Topics Used:

** **AVL Trees:** We use an AVL Tree to organize space objects according to their simulated spatial sectors. Instead of checking every object in the simulation, a satellite can first search its nearby sector to find possible threats, making the search more efficient.*

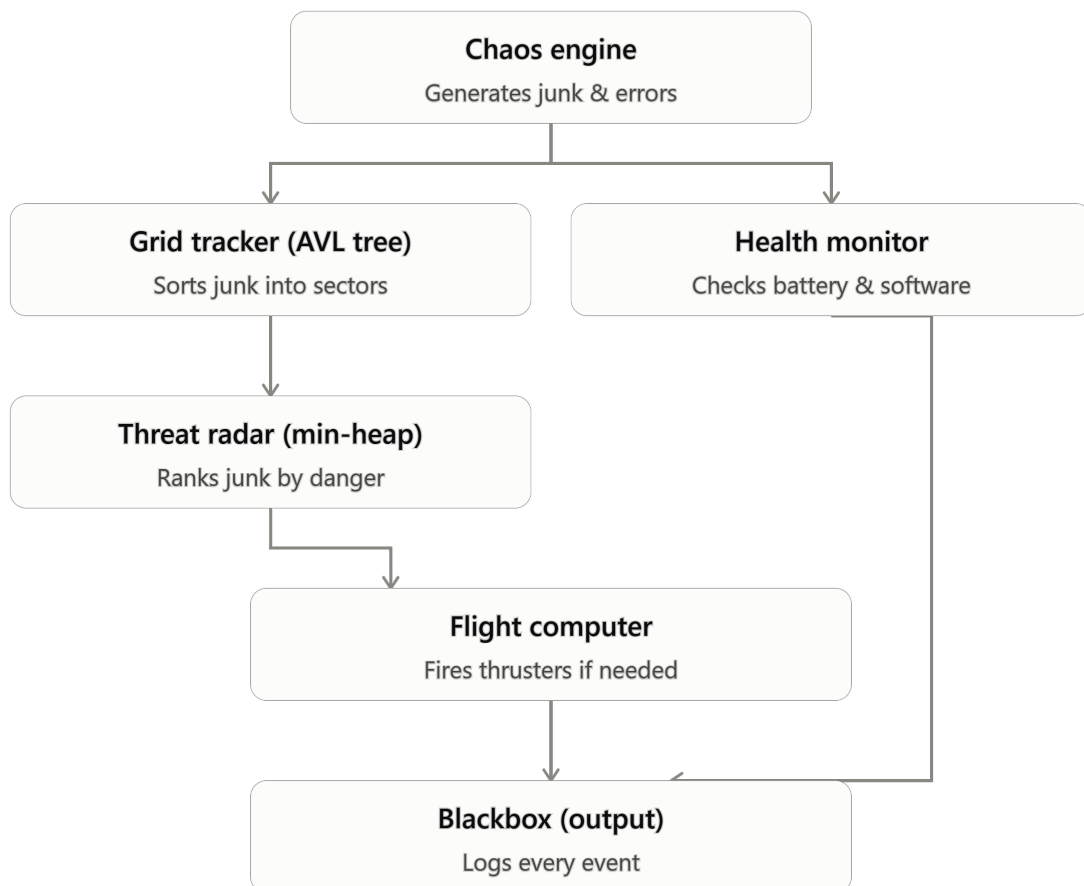
** **Min-Heaps (Priority Queues):** Once a satellite spots nearby junk, it puts the distances into a Min-Heap. This guarantees that the absolute closest, most dangerous piece of junk is automatically pushed to the top of the list so it can be dodged first.*

** **Circular Queues:** We use a circular queue to track a rolling history of the satellite's battery life over time, dropping the oldest data as new data comes in.*

** **Risk and Decision System:** The system combines collision-threat information with the satellite's current health status to decide how the satellite should respond. For example, a healthy satellite with enough fuel may perform an avoidance maneuver, while a satellite with a damaged thruster or critically low fuel may enter a safe mode and generate an emergency alert.*

System Architecture (High Level Diagram)

- * **1. Chaos Engine (Input):** A background system that randomly generates space junk and throws system errors (like battery drains) at the satellites.
- * **2. Grid Tracker (AVL Tree):** Sorts all the moving space junk into specific 3D space sectors for quick searching.
- * **3. Threat Radar (Min-Heap):** Looks at the junk in the satellite's sector and puts the closest, most dangerous piece at the very top of the priority list.
- * **4. Flight Computer (OOP Polymorphism):** Checks the Threat Radar. If junk crosses the safe distance, it fires the thrusters to move the satellite and uses up fuel.
- * **5. Health Monitor (Circular Queue):** Constantly tracks battery history and software. It auto-fixes software bugs or turns the satellite off if a physical part breaks.
- * **6. Blackbox (Output):** Saves every movement, error, and dodge into a text file for review later.



Project Outcome / Deliverables

We will deliver a working C++ program that runs in the computer terminal. While it runs, the screen will show a live update of all satellites, displaying their current health, and printing alert messages if they are dodging debris or fixing bugs. When the simulation finishes, the program will create a blackbox.txt file. This file will be a complete history book of the mission, listing every single event, crash alert, and repair that happened, along with the exact time it occurred.

Assumptions

We assume that satellites can instantly fire their thrusters to change direction. We are using simple straight-line 3D math for movement instead of real-world curved gravity orbits. We also assume the satellites have perfect sensors and know exactly where the space junk is in their immediate AVL Tree sector. Finally, we assume physical repairs take a fixed amount of time to complete. The system is a simplified simulation intended to demonstrate C++ OOP and DSA concepts. It does not model real orbital mechanics, sensor uncertainty, certified spacecraft software, or actual spacecraft thruster control.

References

**C++ coding guides for building self-balancing AVL Trees and Min-Heaps.*

**C++ standard documentation for Object-Oriented Programming principles (Inheritance and Polymorphism).*

** Basic 3D math for calculating the distance between two moving points in space. *A causal graph-based framework for satellite health monitoring – Jie Meng, Jiji Cai, Liang Chang. 2023 IEEE International Conference on Prognostics and Health Management (ICPHM).*

**A Unified Approach to Separation Assurance and Collision Avoidance for Flight Management Systems – Subramanian Ramasamy, Roberto Sabatini, Alessandro Gardi. RMIT University.*

**Collision Risk Management in Algerian Space Agency – Mohamed Kameche, Nadir Bouanani. 2013 IEEE Conference.*

**Satellite Collision Detection using Spatial Data Structures – Christian Hellwig, Fabian Czappa, Martin Michel, Reinhold Bertrand, Felix Wolf. 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS).*